

---

## STEP 1: 1-BIT OPERATIONS

---

BLUE GIRAFFES

**Connor Nauert, Daniel Okoh, James Webb**

**CS.3339 Computer Architecture**

**Texas State University**

September 17, 2026

## 1 Introduction

In this project, we are designing a small computer from scratch by using Verilog for hardware programming. We will also be using GTKWave to analyze test case waveforms and display the inputs and outputs of the gates.

Step 1 involves constructing the initial simple 1-bit circuits.

## 2 Verilog Code

In this section, we will go over every circuit and the associated Verilog code for each one, including the test modules.

### 2.1 Not Circuit

The Not circuit is the most simple of the 1-bit gates. One input, A, is inverted to an output X. If A is a 1, X will be a 0. If A is 0, X will be a 1.

```
1 module not_test (A, X);
2
3     input A;
4     output X;
5
6     assign X = !A;
7
8 endmodule
```

Testing the Not circuit involves creating a register A and measuring an output X. There are two possible states to be tested: when A is 0 and when A is 1. The pound symbol in the snippet below calls for the computer to wait for some amount of nanoseconds, which allows us to observe how the gate acts over time.

```
1 `timescale 1ns / 1ns
2 `include "not_test.v"
3
4 module not_tb;
5
6     reg A;
7     wire X;
8
9     not_test uut(A, X);
10
11     initial begin
```

```

12
13 $dumpfile("not_tb.vcd");
14 $dumpvars(0, not_tb);
15
16 A = 0;
17 #20;
18
19 A = 1;
20 #20;
21
22 $display("Testing not");
23
24 end
25
26 endmodule

```

## 2.2 Nand Circuit

The Nand circuit is composed of a Not gate and an And gate. Two inputs, A and B, are input into an And gate. The output of the And gate is then inverted and output to X. This means that if A and B are both 1, the And gate will invert the combined 1 into a 0. The other three states will output a 1.

```

1 module nand_test (A, B, X);
2
3     input A, B;
4     output X;
5
6     assign X = !(A && B);
7
8 endmodule

```

The Nand circuit requires us to test for four separate states. A and B are taken in as inputs and set to a pattern. We then wait ten nanoseconds, and repeat for all four states.

```

1 `timescale 1ns / 1ns
2 `include "nand_test.v"
3
4 module nand_tb;
5
6 reg A, B;
7 wire X;
8

```

```
9 nand_test uut(A, B, X);
10
11 initial begin
12
13     $dumpfile("nand_tb.vcd");
14     $dumpvars(0, nand_tb);
15
16     A = 0;
17     B = 0;
18     #10;
19
20     A = 0;
21     B = 1;
22     #10;
23
24     A = 1;
25     B = 0;
26     #10;
27
28     A = 1;
29     B = 1;
30     #10;
31
32     $display("Testing nand");
33
34 end
35
36 endmodule
```

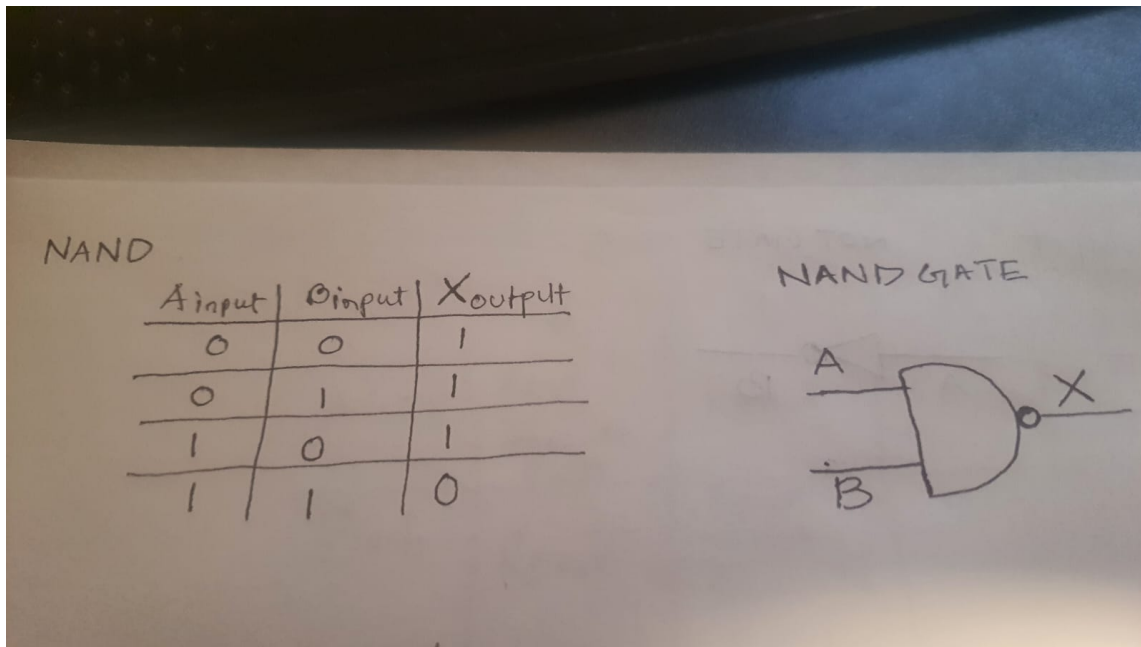


Figure 1: Nand truth table and gate

### 2.3 Nor Circuit

The Nor circuit is similar to the Nand circuit in that it is the inverse of another gate. The Or gate will take two inputs, A and B. The Not gate will switch the bit output by the Or gate, which is then wired to X. If both A and B are 0, then X should be 1. Otherwise, X is 0.

```

1 module nor_test (A, B, X);
2
3     input A, B;
4     output X;
5
6     assign X = !(A || B);
7
8 endmodule

```

The Nor circuit testbench also uses two inputs, one output, and four possible states.

```

1 `timescale 1ns / 1ns
2 `include "nor_test.v"
3
4 module nor_tb;
5
6     reg A, B;
7     wire X;

```

```

8
9 nor_test uut(A, B, X);
10
11 initial begin
12
13     $dumpfile("nor_tb.vcd");
14     $dumpvars(0, nor_tb);
15
16     A = 0;
17     B = 0;
18     #10;
19
20     A = 0;
21     B = 1;
22     #10;
23
24     A = 1;
25     B = 0;
26     #10;
27
28     A = 1;
29     B = 1;
30     #10;
31
32     $display("Testing nor");
33
34 end
35
36 endmodule

```

## 2.4 1x4 Arithmetic Shift Circuit

The Arithmetic Shift circuit will take a 4-bit pattern, and *shift* them over to the left or to the right one bit. Due to this shifter being arithmetic, new bits will be zeros unless shifting a negative integer to the right. Right-shifted negative numbers will be sign-extended, and new bits will be ones.

We implement this by using an array of 4-bit registers as input, which is then mapped to a 4-bit wire output. We also use a *dir* register to specify the direction of the shift.

```

1
2 module bitshift(
3     input signed [3:0] in,

```

```

4     input dir,
5     output signed [3:0] out
6 );
7
8     // If dir is 1, shift in left by 1; otherwise shift in right by 1
9     assign out = dir ? (in <<< 1) : (in >>> 1);
10
11 endmodule

```

This testbench will set a 4-bit pattern, shift to the right, reset, shift to the left, and repeat. For example, a pattern of *1001* shifted to the left would be *0010*, and a pattern of *1010* shifted to the right would be *1101*.

Also, note that *4'b* is used prior to an assignment to tell Verilog we are assigning the bit array with *four binary* values.

```

1  `timescale 1ns / 1ns
2  `include "bitshift.v"
3
4  module bitshift_tb;
5
6  reg signed [3:0] in;
7  reg dir;
8  wire signed [3:0] out;
9
10 bitshift uut(in, dir, out);
11
12 initial begin
13
14     $dumpfile("bitshift_tb.vcd");
15     $dumpvars(0, bitshift_tb);
16
17     in = 4'b0110;
18     dir = 0;
19     #10;
20
21     in = 4'b0110;
22     dir = 1;
23     #10;
24
25     in = 4'b1000;
26     dir = 0;
27     #10;
28

```

```
29   in = 4'b1000;  
30   dir = 1;  
31   #10;  
32  
33   $display("Testing bitshift");  
34  
35 end  
36  
37 endmodule
```

### 3 Waveform Tests

Now, we will showcase waveforms created by testing our four circuits. All waves were generated by vvp and rendered with GTKWave.

#### 3.1 Not Circuit Waveform

In the first half of the waveform, our input (A) is 0, or low. This is inverted to 1 (high) in the output X. After 20ns, A is flipped to 1 and X changes accordingly to 0.

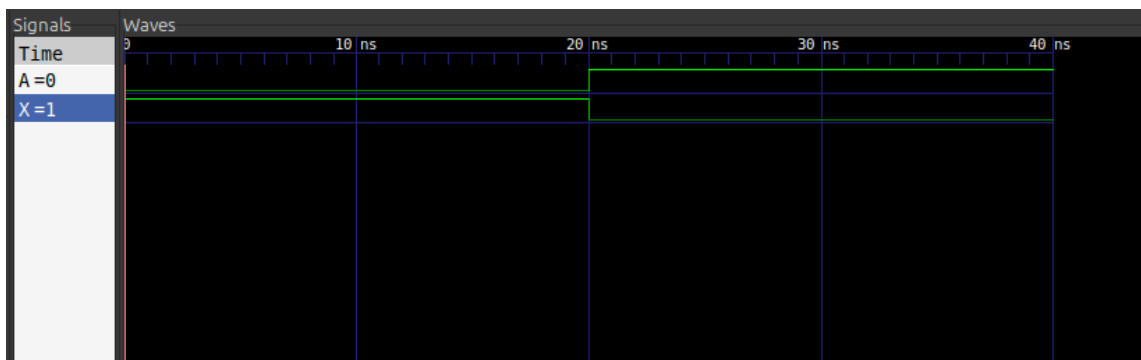


Figure 2: Not circuit waveform. There are two sections since there are two states.

### 3.2 Nand Circuit Waveform

A and B are initially set to 0, and our output X is correctly set to 1. X stays high through two other states until both A and B are set to 1 at 30ns. Since Nand is the inverse of And, X is then set to 0.

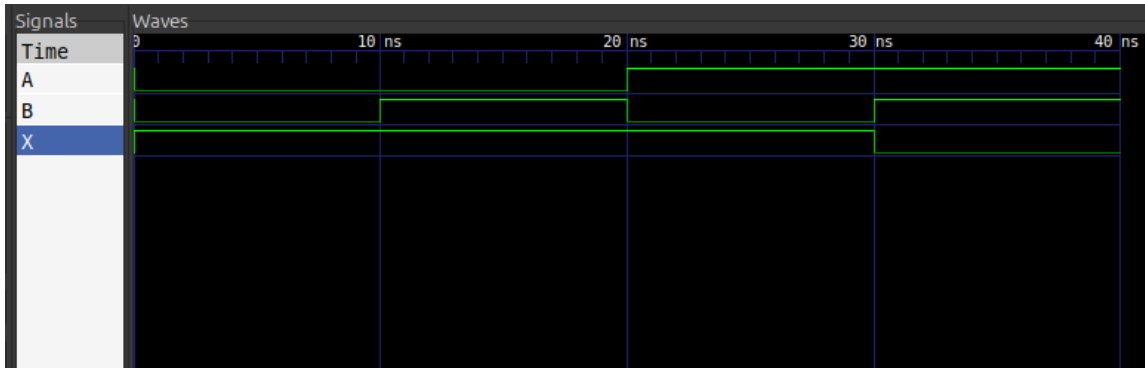


Figure 3: Nand circuit waveform. Notice we now have four states.

### 3.3 Nor Circuit Waveform

Again, A and B are set to 0 at the start of the test. X is high for 10ns since neither A nor B are high. For the remaining 30ns, at least one input is a 1 causing X to become a 0.

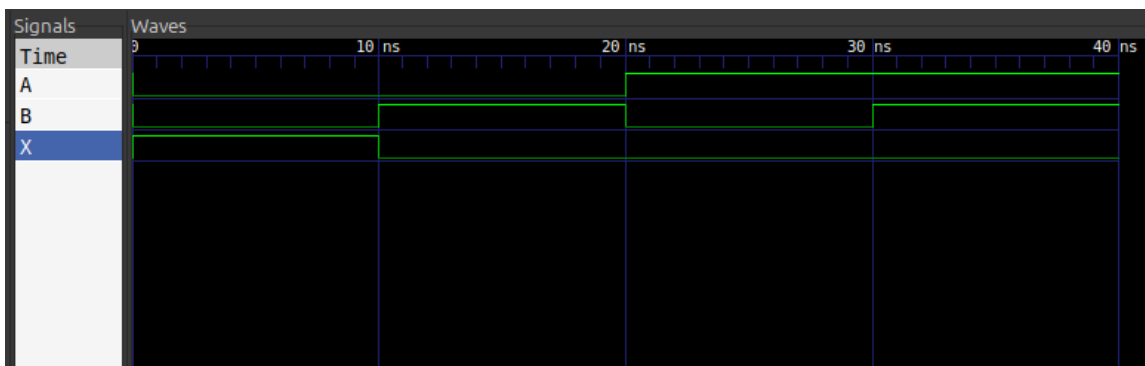


Figure 4: Nor circuit waveform.

### 3.4 1x4 Arithmetic Shifter Waveform

The Arithmetic shifter waveform looks quite complex, but each test case can be explained easily. There are two 4-bit arrays: one for an input and another for the shifted output. On the top, there is a direction register that shifts to the right when 0 and to the left when 1.

At the start, the input array reads *0110*, and the direction reads *0*. This means that we will shift the input to the right one bit. Sure enough, the output is correctly set to the shifted pattern of *0011*. When the direction bit flips to left after 10ns, the output becomes *1100*.

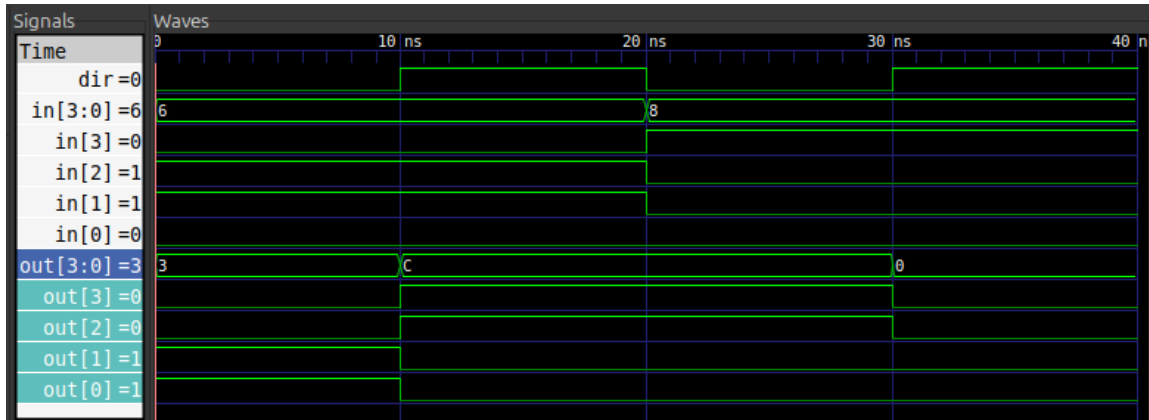


Figure 5: Arithmetic shifter waveforms, with a DIR register and 8 bits.

## 4 Conclusion

We have successfully created four 1-bit circuits in Verilog, tested them under specific inputs, and observed their behavior through waveforms in GTKWave.

We found that the most difficult part of this project was learning how Verilog source files were properly structured and compiled. Luckily, there was a plethora of online documentation and examples on the internet to learn from. LaTeX was helpful as the typesetting was done for us and we could focus on the content of the report. We used Git to collaborate and track changes. We trust that the next few project steps will go smoother considering everything we have learned.